第一阶段-STM32-阶段 2

25 年 11 月 10 日-25 年 11 月 16 日，STM32 点亮 LED、外部中断、定时器

（严格要求了提交方式以及提交格式的，详细请看最下面）

<==========>

STM32—点亮 LED—GPIO 操作

简介：入门嵌入式的第一步，点亮一个 LED。和 C 语言的“Hello World！”一样重要。

基础任务：
1.使用 HAL_GPIO_WritePin 点亮一个 LED;然后 500ms 后将其关闭;
2.使用 HAL_GPIO_TogglePin 翻转 IO 口,每 1000ms 反转一次.
3.使用 HAL_GPIO_ReadPin 检测某个 IO 口电平，可以用按钮控制 IO 口电平，也可以用导线控制电平。如果是低电平，就使小灯点亮，高电平就使小灯熄灭。

STM32—中断

简介：中断系统是单片机实现实时响应和高效多任务处理的核心机制，当某个特定事件（称为中断源）发生时，NVIC(嵌套向量中断控制器)会处理这些中断源，并通知 CPU 暂停当前正在执行的程序，转去执行相应的中断服务程序，完毕后自动返回原程序继续执行。
中断系统的工作原理是基于中断请求、中断优先级和中断向量表的协同工作，当中断源（如外部引脚电平变化、定时器溢出、串口接收数据等）产生中断请求时，如果该中断未被屏蔽且优先级足够高，CPU 就会保存当前状态并跳转到对应的中断服务程序。
中断的关键概念包括中断源（触发中断的事件类型）、中断优先级（决定多个中断同时发生时的处理顺序）、中断使能（开关特定中断）和中断标志（标识中断是否发生）；在实际应用中，中断主要用于实现对外部事件的快速响应、提高 CPU 工作效率、处理突发情况以及协调多个外设的并行工作，是单片机从被动轮询转向主动响应的关键技术突破。

基础任务：
1. 配置外部按键中断，当按键按下时立即触发中断服务程序，实现 LED 状态的即时翻转，掌握外部中断的基本配置和响应处理流程；
2. 思考中断的意义，为什么我们需要中断。

STM32—TIM 定时器/PWM

简介：定时器是 MCU 中最重要的外设之一，硬件本质上是一个值自动递增或递减的寄存器（计数器）。它不依赖 CPU 延时，可以独立、精确地完成与时间相关的操作，同时让 CPU 解放出来去执行其他任务。
这种机制让单片机能够"同时"处理多个任务；定时器的关键组成部分包括时钟源（提供计数基准）、预分频器（调节计数速度）、自动重载值（设定定时周期）和计数模式（向上/向下计数）；在实际应用中，定时器主要用于产生精确延时、创建 PWM 信号控制外设、定时执行任务以及测量外部信号特征，是单片机从简单顺序执行迈向复杂多任务处理的关键技术。

基础任务：
1.使用定时器精准闪烁 LED。不能使用 delay()函数，利用定时器中断配置，让 LED 灯以精确的周期闪烁
2.呼吸灯。使用定时器控制 PWM 波形，控制 LED 产生平滑的呼吸灯效果。
3.定时器与外设协同工作。使用定时器创建定时产生的中断 ？？？

<==========>

对于简单的学习，看看网上的教程然后跟着做，最后实现效果就行了。不会的就先思考再上网问。如果你好奇为什么的话就去深究。

32 学习基本上没有附加任务，也可以说都是附加任务。嵌入式学习基本没有固定套路，只有一个模糊的方向。一切的一切都需要学习者自己去探索，去自己解决问题。
基本任务只是指向了一个大概的方向。在实际检查任务的时候你可以自己发挥想像利用现有资源完成一些事情，只要是和那个方向对应的就行。
不太精通也没关系，和我们的理解不一样也没关系，但是一定要有一套自己的说法。我们也会给出一个大概的说法提供给你参考。纯好玩嘛。

<==========>

以上任务基本上都是单片机的底层逻辑。就像游戏一样，这是开发者的最基本玩法之一。

以下是个人理解：
对于单片机来说，就是一个不断在执行任务的机器（while（1）；）。当一些要紧的事情发送的时候，需要打破死循环转而处理这些事情。这些要紧的事情可以是外部 IO 口的高低电平转换，可以是定时器达到计数值了，也可以是串口收到东西了等等，而这些都是你可以去设置的。
当单片机被打断后，会进入一些特殊的函数，在这个函数里面就可以放一些处理要紧事情的机制。但是有这么多打断的机制，单片机是怎么知到是谁打断它了呢？NVIC，中断处理机制。当单片机被任意的中断打搅后，NVIC 就会开始工作，逐个对比中断源。在寻找到正确的触发者后就会进入到对应的中断回调函数中，也就是这些特殊的函数。

以外部中断来说，最基本的触发模式有上升沿触发，下降沿触发，双边沿触发。上升沿触发就是以前你检测的 IO 口是低电平，突然变为高电平了，然后被单片机捕捉到最后触发中断。其他两种以此类推。
当触发外部中断后，NVIC 就开始检测是哪一个 IO 口触发的呢？比如就是 PA1 吧。对于 STM32 来说，每一组 IO 口都可以是外部中断。这里比较难懂，就是 PB1 和 PA1 其实是一个中断，IO 口编号由字母和数字组成，A、B、C 等这种字母就算是独立的一组，后面的数字就是这一组下的编号。也就是说，简单理解外部中断，有多少个不同的数字就最多有多少和外部中断。对于 STM32 来说，0-4 有单独的中断号例如 EXTI0_IRQn，5-9 共享一个，10-15 共享一个。

然后就是定时器。其实定时器最好理解了。首先要记住的就是定时器其实就是计数器。

例如现在每秒你可以数 100 个数，但是我现在要求你每数 2 个数就写一次自己的名字，然后每写 10 次自己的名字就写一次自己的年龄。理所应当的在 1 秒后你就写了 50 次你自己的名字，写了 5 次自己的年龄。
在这个例子中，**每秒 100 次**就是数数的基本时钟（CLK），**每数 2 个**就是预分频器（PSC），在每数 2 个的基础上**再数 10 次**就是自动重装载值（ARR）。
基本时钟就是数数的速度。
预分频器将基本时钟分为很多份，以前数一个就算一个，现在是数两个算一个。
自动重装载值就是数到多少多少就停止。
那现在想想，假如我现在每秒可以数 72_000_000（72M）个数字，还是需要每秒写 50 次我的名字，写 5 次自己的年龄，那么我的 PSC 和 ARR 需要是多少？先想想先想想。

是不是就是 72000000/50=1440000=PSC，50/5=10=ARR

PSC=1440000，似乎有点大了。。。假设现在条件改变一下，我不需要写自己的名字，只要求每秒输出 5 次自己的年龄。那么合理的 PSC 和 ARR 多少比较好？

72000000/7200=10000=PSC，7200/5=1440=ARR。这个数怎么样？当然很好，结果也不唯一。

PSC=10000，ARR=1440。对其解释就是以前数 1 个算 1，现在数 10000 个才算 1；要求数到 1440 才完成。是的，这就是定时器的最基本原理。

回过头来看 PSC=1440000，ARR=10 和 PSC=10000，ARR=1440，虽然这两组数据都可以完成，为什么我说第二组更好？对于基本定时器来说，最大的定时器记数寄存器是 16 位的，也就是 65536。所以 PSC 和 ARR 都不能超过这个值。对于高级定时器来说，其可以突破，这里不细说。

我希望这能够帮你理解定时器是计数器这句话。且一般用作计数器的时候更多。

刚刚数数的例子都是从 0 数到计数器最大值，这称作向上记数。STM32 也支持向下记数等模式，这里均以向上记数为例子。当记数到最大值时，STM32 就会触发定时器溢出中断，这个时候可以选择对于定时器计数值归零等操作。以刚才的例子来说，也就是完成了一个 200ms 触发一次的中断。

要是我现在不想在记数到最大值的时候就触发中断，我想在记到 75%的时候触发中断，让我的某个 IO 口从低电平变为高电平，在计数值到 75%之前都是保持低电平。

让我们再回到刚才的数数的例子。假设就是 PSC=10000，ARR=1440。1440*0.75=1080，当我们数数数到 1080 的时候，就让 IO 变为高电平吧。数到最大值的时候又归为 0 重新数，重复这个过程。之前在 1 秒钟内我们可以完整的数到最大值数 5 次，且 5 次的每一次中前 75%都是低电平，后 25%都是高电平，在宏观看来这 1 秒钟还是 75%都是低电平，25%是高电平。假设标准 IO 输出高电平是 5V，那么宏观看来这个 IO 口就是 5*0.25=1.25V。这就是 PWM。

但是对于 PSC=1440000，ARR=10 这个极端的例子。10*0.75=7.5。明显我们的寄存器里是不能存直接的小数的。所以 ARR=10 的精度达不到。也就是说 PWM 的精度取决于 ARR。

反过来再看这个例子。假如说 IO 高电平输出还是 5V，我需要 1.25V，是否其他的 PSC 和 ARR 组合也可以达到这个效果呢？

宏观上来看，只需要我的（高电平占总周期的时间之比）占空比=1.25/5=0.25 就可以了对吧。的确如此。但是我再要求你每秒钟需要出现 10 次高电平呢？也就是每秒需要将完整的记数次数达到 10 次！我们试着来计算一下。还是之前定时器的理解方法。72000000/7200=10000=PSC，7200/10=720=ARR，并且使触发高电平输出的记数（TRIG_IN）改成 720*0.75=540。问题完美解决。

PSC=10000，ARR=720，TRIG_IN=540；PSC=10000，ARR=1440，TRIG_IN=1080。这两组数据都可以完成输出 1.25V。但是有何不同？在脑海里面仔细思考一下在这神奇的 1 秒钟都发生了什么？!!

是周期!!!方案一在 1 秒内完整地数了 10 次，拥有 10 个方波；方案二在 1 秒内完整地数了 5 次，拥有 5 个方波。也就是说方案一的单个方波周期 T=100ms，方案二的 T=200ms。这就是区别。这就是改变 ARR 带来的波形上面的效果。ARR 与周期长度直接挂钩。TRIG_IN 配合 ARR 与占空比挂钩。PSC 被动配合 ARR 完成任务。PWM 三个关键参数的关系大概就是这样。

PWM 输出和定时器精密关联，也存在电平定义和向上向下记数。这样都可以像上面这么理解，只是可能场景不太一样。

以上就是我自己对定时器，中断，PWM 的理解，其实很简单，也很好懂。且对于时钟树，我希望你们还是多去学习学习。和我的理解不同也没关系，有自己的理解且能够解释通就行。

<==>

有关代码风格，从现在起就要开始要求了。

这里简单的说一下。

1.#define 的东西需要全大写

例如：`#define MPU6500_REG_GYRO_CONFIG_FS_SEL_250      0x00`

2.函数名称最好是"主要对象_作用_对象"且每一块的第一个字母为大写

例如：`uint8_t MPU6500_Read_Accel(struct MPU6500_Data *mpu6500_data)`

3.全局变量名首单词字母小写，后面的单词首字母大写

例如：`nmeaTIME beiJingTime;`

4.局部变量名小写

例如：`HAL_StatusTypeDef res;`

5.Tab 的空格数为 4 个

6.注释风格

```
/*brief  读取加速度数据
 * @param  mpu6500_data: MPU6500 数据结构体指针
 * @retval 0:成功, 1:失败
 */
uint8_t MPU6500_Read_Accel(struct MPU6500_Data *mpu6500_data)
```

7.关于大括号

```
// 1.大括号对齐
// 大括号保证在同一列,包括 if,for 等大括号
int main(void)  // 空传参要写 void
{

    return 0;
}

// 这样看着很难受,不推荐
int main(void){

    return 0;
}

// 2.大括号省略
// 对于 if 和 for 等非函数大括号,如果只有一句话你可以提行不写大括号
if(a == b)
    return 0;

// 这样也可以
if(a == b)  return 0;

// 这样也可以
if(a == b)
{
    return 0;
}
```

比较重要的就这些了，后续还会根据情况添加。这个东西是约定俗称的，是为了大家之间代码看的更加清楚。当然这个也不是最规范的，也应该不存在最规范这一说。

<==========>

25 年 11 月 16 日（周天）下午两点到六点到实验室检查打分。带上你的成品电路和电脑，必要时可能会检查代码。